



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks

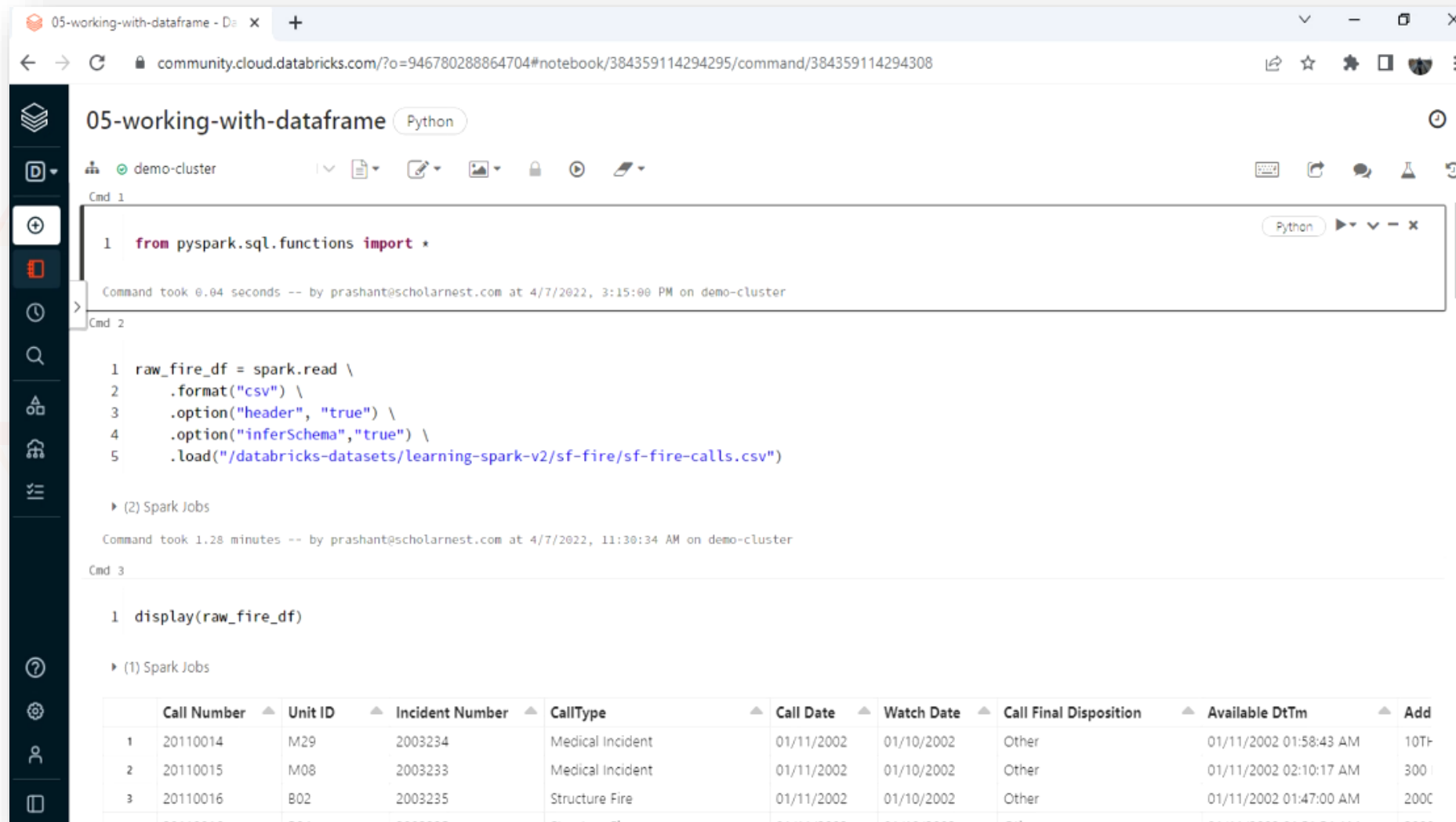


Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Working with Spark Dataframe – Part I

Go to your Databricks workspace and open the notebook we created in the previous lecture. **(Reference: 05-working-with-dataframe.ipynb)**



05-working-with-dataframe Python

demo-cluster

Cmd 1

```
1 from pyspark.sql.functions import *
```

Command took 0.04 seconds -- by prashant@scholarnest.com at 4/7/2022, 3:15:00 PM on demo-cluster

Cmd 2

```
1 raw_fire_df = spark.read \  
2   .format("csv") \  
3   .option("header", "true") \  
4   .option("inferSchema", "true") \  
5   .load("/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv")
```

► (2) Spark Jobs

Command took 1.28 minutes -- by prashant@scholarnest.com at 4/7/2022, 11:30:34 AM on demo-cluster

Cmd 3

```
1 display(raw_fire_df)
```

► (1) Spark Jobs

	Call Number	Unit ID	Incident Number	CallType	Call Date	Watch Date	Call Final Disposition	Available DtTm	Add
1	20110014	M29	2003234	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 01:58:43 AM	10TH
2	20110015	M08	2003233	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 02:10:17 AM	300
3	20110016	B02	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
4	20110016	B04	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:51 AM	200C

We have transformed the created a Dataframe fire_df which is ready for analysis. You can see the fire_df Dataframe here in the screenshot below.

05-working-with-dataframe Python

demo-cluster

```
4 .withColumn("AvailableDtTm", to_timestamp("AvailableDtTm", "MM/dd/yyyy hh:mm:ss a")) \
5 .withColumn("Delay", round("Delay", 2))
```

Command took 0.15 seconds -- by prashant@scholarnest.com at 4/7/2022, 3:16:56 PM on demo-cluster

Cmd 8

```
1 display(fire_df)
```

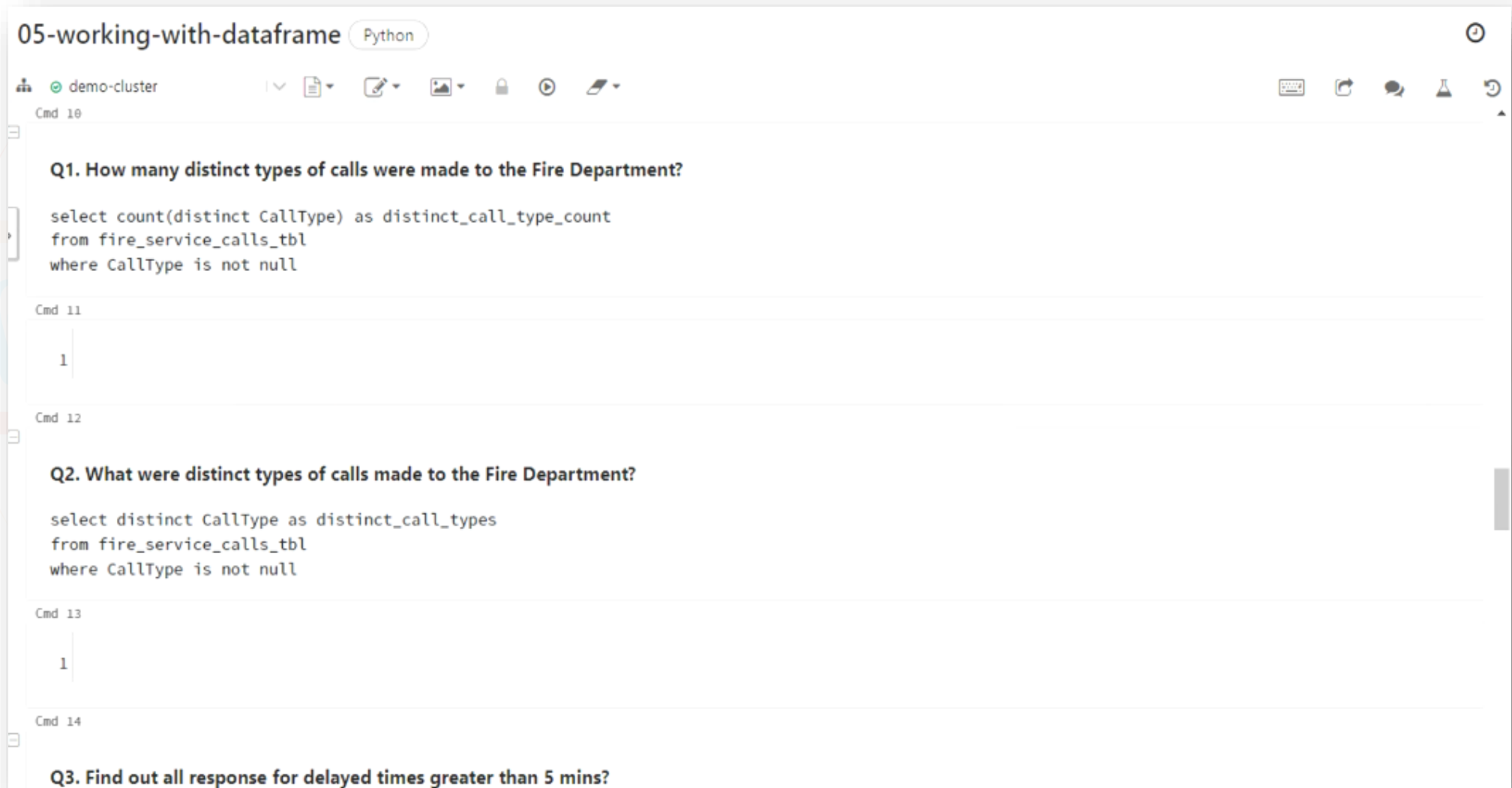
▶ (1) Spark Jobs

	CallNumber	UnitID	IncidentNumber	CallType	CallDate	WatchDate	CallFinalDisposition	AvailableDtTm
1	20110014	M29	2003234	Medical Incident	2002-01-11	2002-01-10	Other	2002-01-11T01:58:43.000+0000
2	20110015	M08	2003233	Medical Incident	2002-01-11	2002-01-10	Other	2002-01-11T02:10:17.000+0000
3	20110016	B02	2003235	Structure Fire	2002-01-11	2002-01-10	Other	2002-01-11T01:47:00.000+0000
4	20110016	B04	2003235	Structure Fire	2002-01-11	2002-01-10	Other	2002-01-11T01:51:54.000+0000
5	20110016	D2	2003235	Structure Fire	2002-01-11	2002-01-10	Other	2002-01-11T01:47:00.000+0000
6	20110016	E03	2003235	Structure Fire	2002-01-11	2002-01-10	Other	2002-01-11T01:47:00.000+0000
7	20110016	E38	2003235	Structure Fire	2002-01-11	2002-01-10	Other	2002-01-11T01:51:17.000+0000

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)

Command took 1.98 seconds -- by prashant@scholarnest.com at 4/7/2022, 3:17:24 PM on demo-cluster

We can document all the 10 micro project question in this notebook along with it's solution SQL expression which we have already seen in the earlier lectures.



The screenshot shows a Jupyter Notebook interface with the title '05-working-with-dataframe' and a 'Python' language selector. The notebook is running on a 'demo-cluster'. It contains four command cells, labeled 'Cmd 10' through 'Cmd 14'. The first three cells contain SQL queries and their results. The fourth cell contains a question but no query or result is visible.

```
Cmd 10

Q1. How many distinct types of calls were made to the Fire Department?

select count(distinct CallType) as distinct_call_type_count
from fire_service_calls_tbl
where CallType is not null

Cmd 11

1

Cmd 12

Q2. What were distinct types of calls made to the Fire Department?

select distinct CallType as distinct_call_types
from fire_service_calls_tbl
where CallType is not null

Cmd 13

1

Cmd 14

Q3. Find out all response for delayed times greater than 5 mins?
```

Before I start analysis, let me cache the Dataframe. You can use the `cache()` utility method to cache the Dataframe into memory. Because I want to run ten analysis transformations on the same Dataframe.



The screenshot shows a Databricks notebook interface. The notebook title is "05-working-with-dataframe" and the language is set to "Python". The notebook is running on a cluster named "demo-cluster". The code cell contains the command `fire_df.cache()`. The output shows the schema of the DataFrame, which includes columns like CallNumber, UnitID, IncidentNumber, CallType, CallDate, WatchDate, CallFinalDisposition, AvailableDtTm, Address, City, Zipcode, Battalion, StationArea, Box, OrigPriority, Priority, FinalPriority, ALSUnit, CallTypeGroup, NumAlarms, UnitType, UnitSequenceInCallDispatch, FirePreventionDistrict, SupervisorDistrict, Neighborhood, Location, RowID, and Delay. The command took 0.15 seconds to execute.

```
05-working-with-dataframe Python

demo-cluster
|-- ALSUnit: boolean (nullable = true)
|-- CallTypeGroup: string (nullable = true)
|-- NumAlarms: integer (nullable = true)
Command took 0.05 seconds -- by prashant@scholarnest.com at 4/7/2022, 9:11:32 PM on demo-cluster

Cmd 10

1 fire_df.cache()

Out[12]: DataFrame[CallNumber: int, UnitID: string, IncidentNumber: int, CallType: string, CallDate: date, WatchDate: date, CallFinalDisposition: string, AvailableDtTm: timestamp, Address: string, City: string, Zipcode: int, Battalion: string, StationArea: string, Box: string, OrigPriority: string, Priority: string, FinalPriority: int, ALSUnit: boolean, CallTypeGroup: string, NumAlarms: int, UnitType: string, UnitSequenceInCallDispatch: int, FirePreventionDistrict: string, SupervisorDistrict: string, Neighborhood: string, Location: string, RowID: string, Delay: double]
Command took 0.15 seconds -- by prashant@scholarnest.com at 4/7/2022, 9:13:44 PM on demo-cluster
```

We already know the logic behind the 10 question as we implemented the same using SQL. But SQL runs on a table. And we want to learn how to run those same 10 logic on a Dataframe.

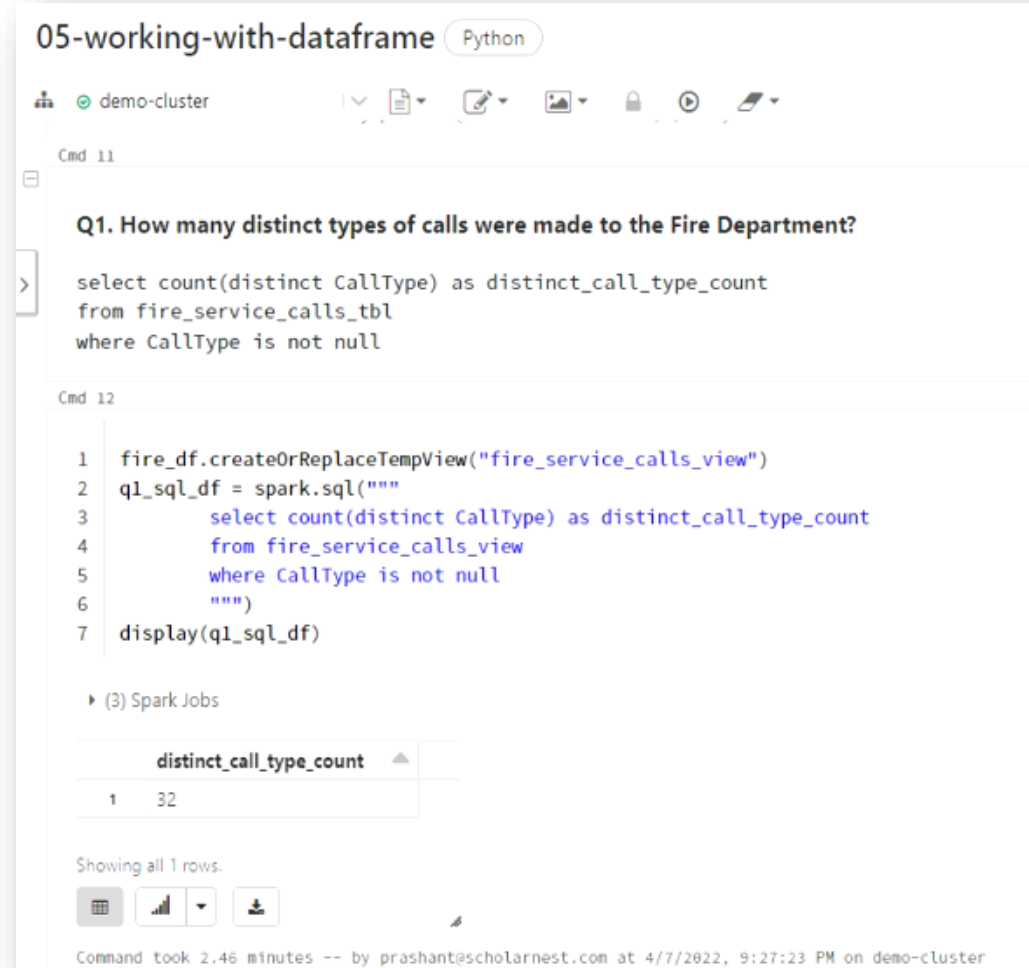
We have two approaches:

1. SQL Approach
2. Dataframe Transformation Approach

Further, the SQL approach is a two-step process:

1. Convert your Dataframe to a temporary view
2. Run your SQL on the view

Let me proceed with the SQL approach, the first thing is to convert my Dataframe to a temporary view. So I am using the same SQL that I used on the table. But I changed the table name to the view name. Now, the second step is to run SQL queries on the view. The second step will give you a Dataframe.



The screenshot shows a Databricks workspace interface. At the top, it says "05-working-with-dataframe" and "Python". Below that, there's a toolbar with various icons. The main area displays two commands. Command 11 is a SQL query: "Q1. How many distinct types of calls were made to the Fire Department?" followed by "select count(distinct CallType) as distinct_call_type_count from fire_service_calls_tbl where CallType is not null". Command 12 is a Python code block: "1 fire_df.createOrReplaceTempView('fire_service_calls_view')", "2 q1_sql_df = spark.sql(''", "3 select count(distinct CallType) as distinct_call_type_count", "4 from fire_service_calls_view", "5 where CallType is not null", "6 '")", "7 display(q1_sql_df)". Below the code, it says "(3) Spark Jobs". A table with one row is shown: "distinct_call_type_count" with the value "32". At the bottom, it says "Showing all 1 rows." and "Command took 2.46 minutes -- by prashant@scholarnest.com at 4/7/2022, 9:27:23 PM on demo-cluster".

```
05-working-with-dataframe Python

demo-cluster

Cmd 11

Q1. How many distinct types of calls were made to the Fire Department?

select count(distinct CallType) as distinct_call_type_count
from fire_service_calls_tbl
where CallType is not null

Cmd 12

1 fire_df.createOrReplaceTempView("fire_service_calls_view")
2 q1_sql_df = spark.sql("""
3     select count(distinct CallType) as distinct_call_type_count
4     from fire_service_calls_view
5     where CallType is not null
6     """)
7 display(q1_sql_df)

(3) Spark Jobs

distinct_call_type_count
1 32

Showing all 1 rows.

Command took 2.46 minutes -- by prashant@scholarnest.com at 4/7/2022, 9:27:23 PM on demo-cluster
```

Now let us see the Dataframe Transformation approach. Here we have the logic for this approach given below. We can build the logic easily just by looking at SQL solutions.

05-working-with-dataframe Python

demo-cluster

Cmd 11

Q1. How many distinct types of calls were made to the Fire Department?

```
select count(distinct CallType) as distinct_call_type_count
from fire_service_calls_tbl
where CallType is not null
```

Cmd 12

```
1 fire_df.createOrReplaceTempView("fire_service_calls_view")
2 q1_sql_df = spark.sql("""
3     select count(distinct CallType) as distinct_call_type_count
4     from fire_service_calls_view
5     where CallType is not null
6     """)
7 display(q1_sql_df)
```

Show result

Cmd 13

1. Filter the records and take only those where CallType is not null
2. Select the CallType column
3. Take only distinct call types
4. Show the count

Here is the code for the Dataframe Transformation approach. We got the same desired output with this approach as well.

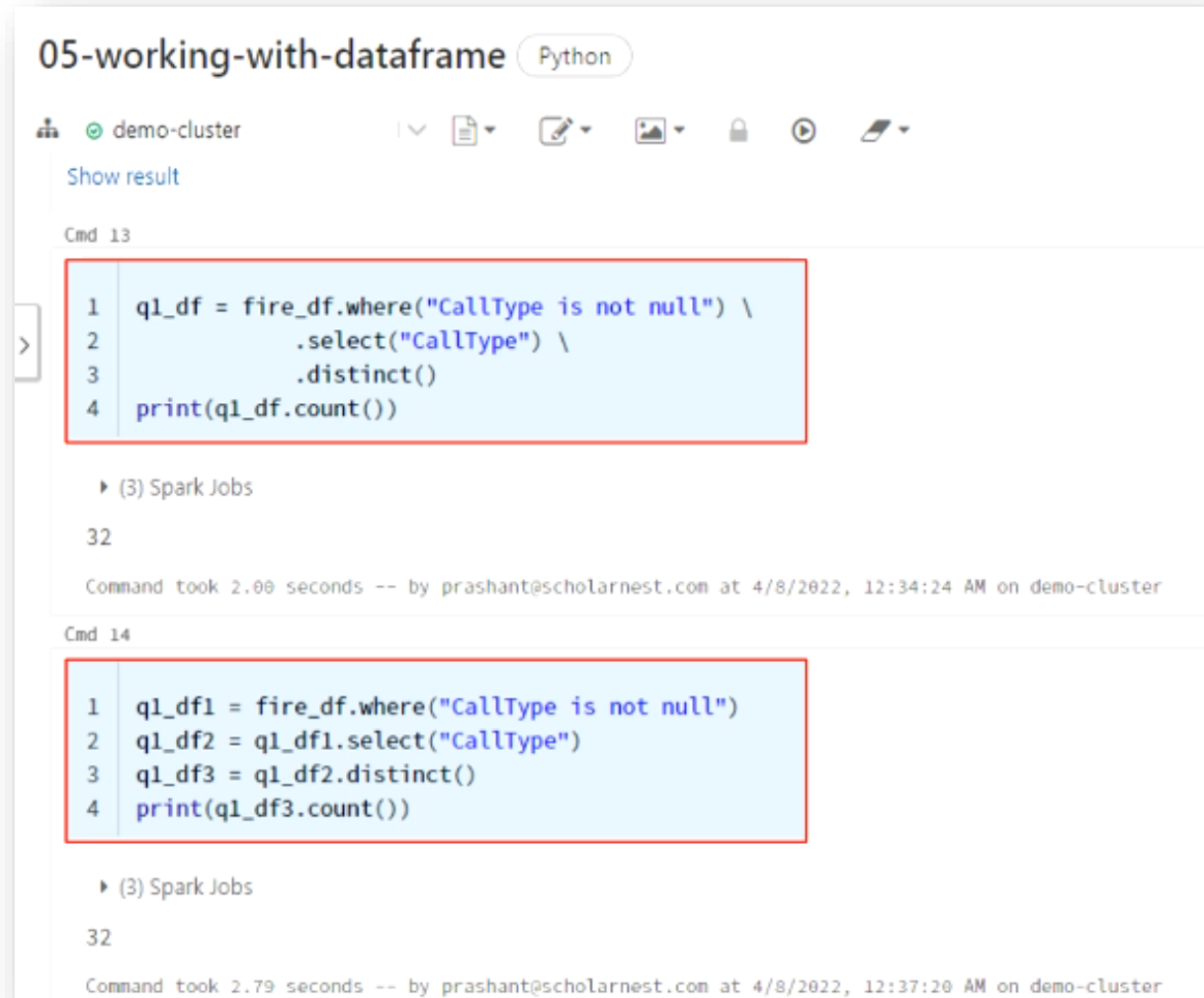


The screenshot shows a Databricks notebook interface. At the top, the notebook name is "05-working-with-dataframe" and the language is set to "Python". Below the name, there's a toolbar with icons for cluster management, file operations, and execution. The cluster name "demo-cluster" is visible. A "Show result" link is present. The command area shows "Cmd 13" with the following PySpark code:

```
1 q1_df = fire_df.where("CallType is not null") \
2     .select("CallType") \
3     .distinct()
4 print(q1_df.count())
```

Below the code, it indicates "(3) Spark Jobs" and the output "32". At the bottom, a status message reads: "Command took 2.00 seconds -- by prashant@scholarnest.com at 4/8/2022, 12:34:24 AM on demo-cluster".

We can also write the code for the Dataframe approach in the manner shown below. Both the approach will give you the same answer, but it is recommended to follow the first approach shown in the image.



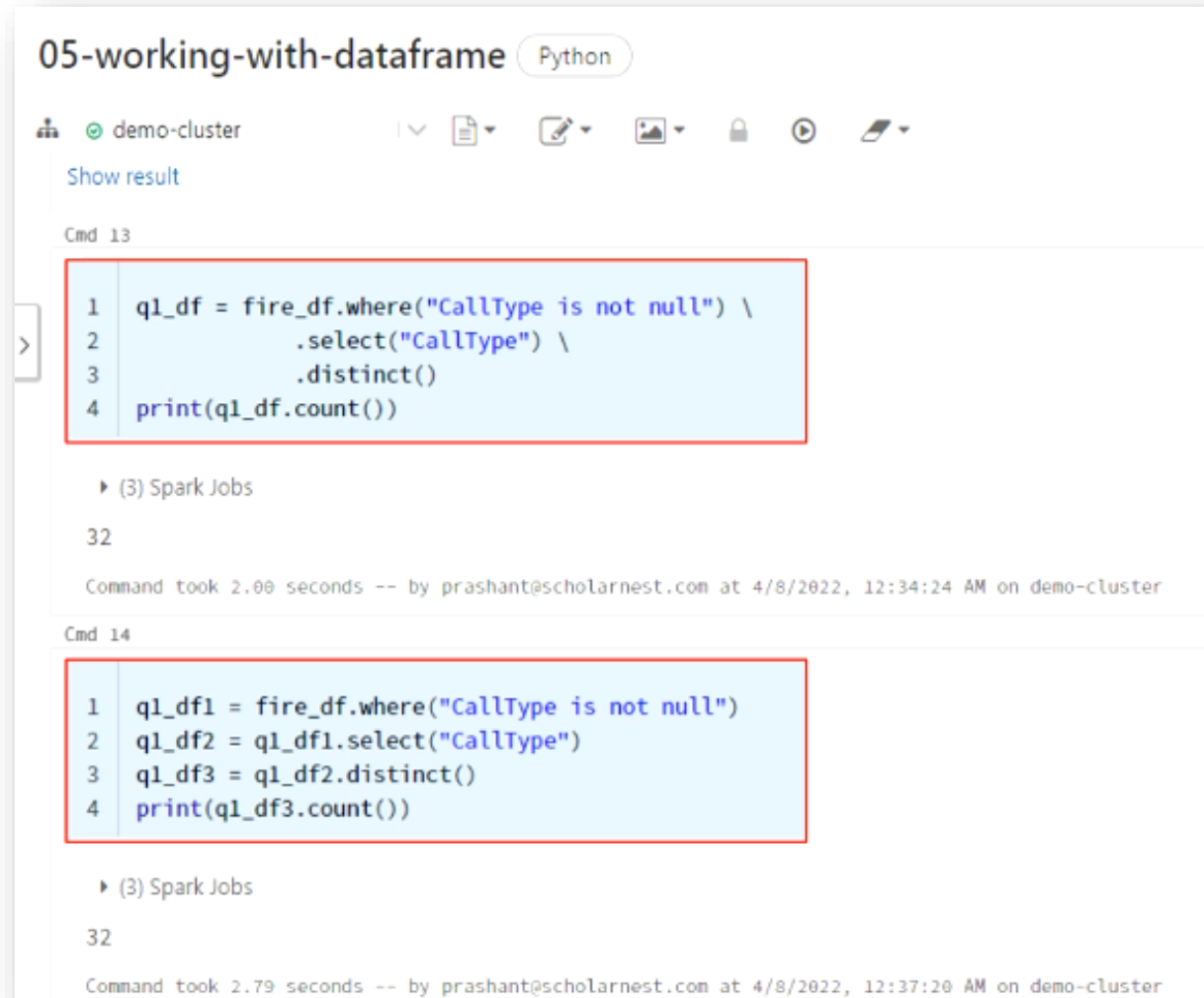
```
05-working-with-dataframe Python
demo-cluster
Show result
Cmd 13
1 q1_df = fire_df.where("CallType is not null") \
2     .select("CallType") \
3     .distinct()
4 print(q1_df.count())

(3) Spark Jobs
32
Command took 2.00 seconds -- by prashant@scholarnest.com at 4/8/2022, 12:34:24 AM on demo-cluster

Cmd 14
1 q1_df1 = fire_df.where("CallType is not null")
2 q1_df2 = q1_df1.select("CallType")
3 q1_df3 = q1_df2.distinct()
4 print(q1_df3.count())

(3) Spark Jobs
32
Command took 2.79 seconds -- by prashant@scholarnest.com at 4/8/2022, 12:37:20 AM on demo-cluster
```

We can also write the code for the Dataframe approach in the manner shown below. Both the approach will give you the same answer, but it is recommended to follow the first approach shown in the image.



```
05-working-with-dataframe Python
demo-cluster
Show result
Cmd 13
1 q1_df = fire_df.where("CallType is not null") \
2     .select("CallType") \
3     .distinct()
4 print(q1_df.count())

(3) Spark Jobs
32
Command took 2.00 seconds -- by prashant@scholarnest.com at 4/8/2022, 12:34:24 AM on demo-cluster

Cmd 14
1 q1_df1 = fire_df.where("CallType is not null")
2 q1_df2 = q1_df1.select("CallType")
3 q1_df3 = q1_df2.distinct()
4 print(q1_df3.count())

(3) Spark Jobs
32
Command took 2.79 seconds -- by prashant@scholarnest.com at 4/8/2022, 12:37:20 AM on demo-cluster
```



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com